

# Stringhe C-style vs string

---

## Stringhe C-style vs string

*La differenza tra le stringhe in stile C (char array) e il tipo string del C++.*

## Le stringhe in stile C

In C++ esistono due modi diversi per lavorare con le parole e le frasi. Il primo è il modo "vecchio", ereditato dal linguaggio C. Il secondo è il modo moderno, molto più semplice.

Devi conoscere entrambi perché li incontrerai nel codice esistente, ma nella pratica ne userai quasi sempre uno solo.

## I due metodi

1. **Stringhe C-style** — sono array di caratteri ( `char[]` ), ereditate dal linguaggio C. Più complicate da usare.
2. **`std::string`** — la stringa moderna del C++, molto più semplice e sicura.

**Regola pratica:** usa sempre `std::string` . Impara le stringhe C-style solo per capire il codice che le usa.

## Le stringhe C-style: array di caratteri

Una stringa C-style è in realtà un array di `char` che termina con un carattere speciale: il carattere nullo `'\0'` . Questo terminatore dice "la stringa finisce qui".

```
char saluto[] = "Ciao";  
// Internamente memorizzata come: {'C', 'i', 'a', 'o', '\0'}  
// Occupa 5 byte (4 lettere + il terminatore)
```

Puoi accedere ai singoli caratteri con l'indice:

```
char nome[] = "Alice";  
cout << nome[0] << endl; // A  
cout << nome[1] << endl; // l
```

## Le funzioni per le C-style strings

Per manipolare queste stringhe hai bisogno di funzioni speciali dalla libreria `<cstring>` :

```
#include <cstring>  
  
char a[] = "Ciao";  
char b[] = "Mondo";  
  
cout << strlen(a) << endl; // 4 – lunghezza della stringa (senza  
'\0')  
strcpy(a, "Hello"); // copia "Hello" in a  
strcat(a, " World"); // concatena " World" ad a  
cout << strcmp("abc", "abc"); // 0 – le due stringhe sono uguali
```

## I problemi delle stringhe C-style

- Non puoi concatenarle con `+`
- Devi gestire la memoria manualmente
- È facile scrivere oltre i limiti del buffer (bug difficile da trovare)
- Il codice risultante è lungo e poco leggibile

```
char a[10] = "Ciao";  
// ERRORE – non puoi fare a + " mondo" con le C-style strings  
// Devi usare: strcat(a, " mondo");
```

## Le stringhe moderne: `std::string`

`std::string` risolve tutti i problemi delle stringhe C-style. Per usarla, includi `<string>` :

```
#include <string>
using namespace std;

string nome = "Alice";
string cognome = "Bianchi";

// Concatenazione semplice con +
string nomeCompleto = nome + " " + cognome;

// Lunghezza con .length()
cout << nome.length() << endl;    // 5

// Confronto diretto con ==
if (nome == "Alice") {
    cout << "Ciao Alice!" << endl;
}
```

Tutto quello che con le C-style strings richiedeva funzioni speciali, con `std::string` si fa con operatori normali.

## Confronto diretto

| Operazione             | C-style ( char[] )                | std::string                     |
|------------------------|-----------------------------------|---------------------------------|
| Dichiarazione          | <code>char s[20] = "ciao";</code> | <code>string s = "ciao";</code> |
| Lunghezza              | <code>strlen(s)</code>            | <code>s.length()</code>         |
| Copia                  | <code>strcpy(dest, src)</code>    | <code>dest = src</code>         |
| Concatenazione         | <code>strcat(a, b)</code>         | <code>a + b</code>              |
| Confronto              | <code>strcmp(a, b) == 0</code>    | <code>a == b</code>             |
| Accesso a un carattere | <code>s[i]</code>                 | <code>s[i]</code>               |

## Convertire tra i due tipi

A volte devi passare da uno all'altro (per esempio, alcune librerie richiedono il tipo C-style):

```
// Da string a char[] (C-style)
string str = "Ciao";
const char* cstr = str.c_str(); // converte in puntatore al C-string

// Da char[] a string - la conversione è automatica
char cArray[] = "Ciao";
string str2 = cArray;
```

## Quando si usano le C-style strings

In pratica moderna, usa sempre `std::string`. Le C-style strings restano necessarie solo in casi particolari:

- Lavorare con librerie scritte in C che richiedono `const char*`
- Programmazione di sistema a basso livello
- Programmi dove ogni byte di memoria è importante

```
// Le librerie C spesso richiedono const char*
// Con std::string usi .c_str() per convertire
string nomefile = "dati.txt";
FILE* f = fopen(nomefile.c_str(), "r");
```

## Esempio: confronto pratico

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

int main() {
    // Approccio C-style – verboso e fragile
    char c_nome[50] = "Alice";
    char c_cognome[50] = "Bianchi";
    char c_completo[100];
    strcpy(c_completo, c_nome);    // copia il nome
    strcat(c_completo, " ");       // aggiunge uno spazio
    strcat(c_completo, c_cognome); // aggiunge il cognome
    cout << "C-style: " << c_completo << endl;

    // Approccio moderno con string – semplice e leggibile
    string s_nome = "Alice";
    string s_cognome = "Bianchi";
    string s_completo = s_nome + " " + s_cognome;
    cout << "string:  " << s_completo << endl;

    return 0;
}
```

Il codice con `std::string` è molto più semplice e difficile da sbagliare.